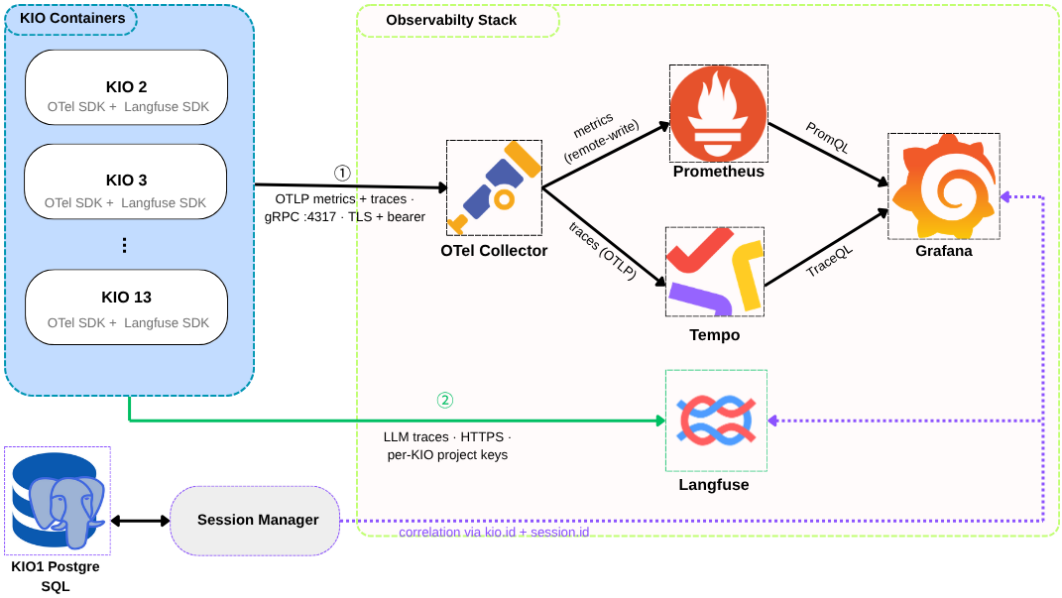
AI4SWENG Observability Integration Contract

This contract is for KIO2-KIO13 owners and defines how each KIO sends telemetry into the AI4SWENG observability stack. Telemetry is push-only, each KIO instruments its own code with the OpenTelemetry SDK, pushes OTLP metrics/traces to the AI4SWENG Collector, and sends LLM traces to that KIO owner's own Langfuse project; KIOs never run their own collector and expose scrape endpoints.

Architecture

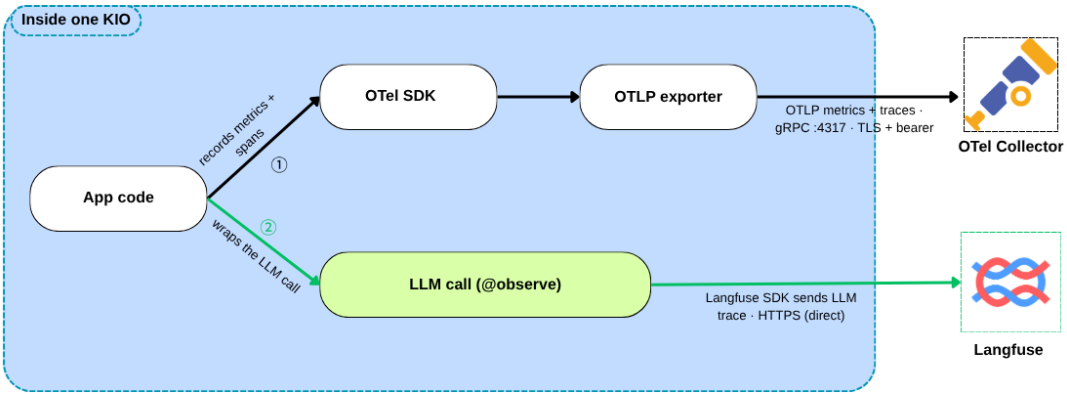
a) High Level

This diagram shows the high-level observability architecture. Each KIO pushes two streams: OTLP metrics/traces to the shared OTel Collector for central dashboards, and LLM traces directly to Langfuse. Session Manager and KIO1 PostgreSQL remain on the platform side, and kio.id + session.id provide correlation/lookup across the observability stack.



b) Low Level

This diagram shows the low-level instrumentation path inside one KIO. App code records metrics/traces through the OTel SDK and OTLP exporter to the shared OTel Collector using TLS and bearer authentication, while LLM calls are wrapped with the Langfuse decorator and sent to Langfuse. Every signal carries service.name, service.version, kio.id, session.id, and deployment.environment.



1- Onboarding & Connectivity Requirements

Follow these steps to integrate with the observability stack:

1. Request credentials: Open a request with the AI4SWEng team for your OTLP bearer token and your own Langfuse project keys.
2. Set the env vars: Collector endpoint, the Authorization: Bearer header, the five resource attributes via OTEL_RESOURCE_ATTRIBUTES, and your LANGFUSE_* keys.
3. Install the SDKs: opentelemetry-sdk, opentelemetry-exporter-otlp-proto-grpc, and langfuse.
4. Attach the resource attributes: service.name, service.version, kio.id, deployment.environment (from the env) and session.id (stamped per run, on your traces).

5. Verification: You are onboarded only once both land on our side: a test metric showing in Grafana and a test trace showing in your Langfuse project.

2- Telemetry Compliance Requirements

Mandatory metrics are listed at the table below:

Metric (dot-style)	Type	Labels	What it means
kio.requests	counter	kio.id	Requests your KIO handled
kio.request.duration	histogram (s)	kio.id	Per-request latency
kio.errors	counter	kio.id, error.type	Requests that failed
kio.llm.tokens	histogram	kio.id, token.type, model	LLM tokens in / out
kio.llm.cost	counter	kio.id, model	LLM cost
kio.sessions.active	up-down counter	kio.id	Sessions in flight right now

Langfuse rule: Emit to your own Langfuse project (isolation/billing) and stamp every trace with kio.id and session.id metadata, which is what lets us aggregate across all KIOs centrally. Keep it to one trace per session, with a span per LLM or tool call.

Export continuously and emit a heartbeat (kio.heartbeat) so we can see your KIO is alive. Beyond the baseline, extra metrics are self-service, as long as it follows the guidelines below.

Reference code and guidelines

```
# env:
# OTEL_EXPORTER_OTLP_ENDPOINT=https://collector.ai4sweng.internal:4317
# OTEL_EXPORTER_OTLP_HEADERS=Authorization=Bearer%20<TOKEN>
# OTEL_RESOURCE_ATTRIBUTES=service.name=<KIO_ID>,service.version=<VERSION>,kio.id=<KIO_ID>,deployment.environment=prod
# LANGFUSE_HOST=... LANGFUSE_PUBLIC_KEY=... LANGFUSE_SECRET_KEY=...

import time
from opentelemetry import metrics, trace
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
from opentelemetry.exporter.otlp.proto.grpc.metric_exporter import OTLPMetricExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from langfuse import observe, get_client, propagate_attributes

KIO_ID = "<KIO_ID>"; BASE = {"kio.id": KIO_ID}; res = Resource.create()

reader = PeriodicExportingMetricReader(OTLPMetricExporter(), export_interval_millis=5000)
metrics.set_meter_provider(MeterProvider(resource=res, metric_readers=[reader]))
m = metrics.get_meter(KIO_ID)
tp = TracerProvider(resource=res); tp.add_span_processor(BatchSpanProcessor(OTLPSpanExporter()))
trace.set_tracer_provider(tp); tracer = trace.get_tracer(KIO_ID)

requests = m.create_counter("kio.requests")
latency = m.create_histogram("kio.request.duration", unit="s")
errors = m.create_counter("kio.errors")
tokens = m.create_histogram("kio.llm.tokens", unit="{token}")
cost = m.create_counter("kio.llm.cost", unit="USD")
sessions = m.create_up_down_counter("kio.sessions.active")
heartbeat = m.create_counter("kio.heartbeat")  # emit continuously from service runtime

@observe(name="kio.session")
def handle(request, *, session_id):
    with propagate_attributes(session_id=session_id, metadata={**BASE, "session.id": session_id}):
        sessions.add(1, BASE); t0 = time.perf_counter()
        with tracer.start_as_current_span("kio.handle") as span:
            span.set_attribute("kio.id", KIO_ID); span.set_attribute("session.id", session_id)
            try:
                requests.add(1, BASE); return answer(request)
            except Exception as e:
                errors.add(1, {**BASE, "error.type": type(e).__name__}); raise
            finally:
                latency.record(time.perf_counter() - t0, BASE); sessions.add(-1, BASE)

@observe(name="llm.generate", as_type="generation")
def answer(request):
    reply = llm.chat(model="<MODEL_NAME>", prompt=request.text)
    labels = {**BASE, "model": reply.model}
    tokens.record(reply.in_tokens, {**labels, "token.type": "input"})
    tokens.record(reply.out_tokens, {**labels, "token.type": "output"})
    cost.add(reply.cost_usd, labels)
    get_client().update_current_generation(
        model=reply.model, usage_details={"input": reply.in_tokens, "output": reply.out_tokens})
    return reply.text
```

Guidelines: Use stable dot-style metric names: kio.<domain>.<metric>. Always include kio.id and session.id where the signal belongs to a session. Do not use unbounded or high-cardinality label values such as raw IDs, emails, or URLs. Never put secrets or PII in metric labels, span names, or Langfuse payloads. Always set service.version. New mandatory metrics are announced by AI4SWEng; optional metrics are self-service if they follow these rules.